PROJECT MANAGEMENT PLAN

Moderated Learning Management System (LMS)

CS4433 - Software Engineering · Individual Assignment

Student: Mweli Pama

Student Number: 201404452

Institution: National University of Lesotho - Department of Mathematics & Computer Science

Phase: Phase 1 - Project Planning and Management (10 marks)

Date: 7 October 2026

Version: 1.0

Development Model: Waterfall with V-Model verification & validation

Document Control

Document title	Project Management Plan - Moderated Learning Management System (LMS)
Course	CS4433 - Software Engineering (Individual Assignment)
Author	Mweli Pama (201404452)
Phase	Phase 1 - Project Planning and Management
Version	1.0 (first submission of the Phase 1 plan)
Date	7 October 2026
Status	Baseline - Phase 1 planning complete; Phases 2–10 are planned future work
Companion file	CS4433_Phase1_Project_Schedule_Mweli_Pama_201404452.xlsx (editable schedule / Gantt master)

This plan is the Phase 1 deliverable. The schedule and Gantt are maintained in the companion Excel master; the Gantt reproduced in Section 4 is exported from that file. Where the two ever differ, the Excel master is authoritative and this narrative is updated to match.

Table of Contents

1. Introduction and Scope 3
2. Feasibility Study 3
    2.1 Technical Feasibility 3
    2.2 Operational Feasibility 3
    2.3 Economic Feasibility 4
    2.4 Legal Feasibility 4
    2.5 Schedule Feasibility 4
    2.6 Overall Recommendation 5
3. Work Breakdown Structure 5
4. Project Schedule 6
    4.1 Dependencies 6
    4.2 Milestones 7
    4.3 Critical Path 7
5. Risk Management 7
    5.1 Risk Method 7
    5.2 Phase 1 Baseline Risk Register 7
    5.3 Risk Review and Gate Updates 8
6. Assumptions and Constraints 8
7. Gate-Review Governance 9
References 9

1. Introduction and Scope

This document commits the Moderated Learning Management System (LMS) to delivery in seven weeks under a gated Waterfall process with V-Model verification and validation, and it recommends proceeding (Section 2.6).

The system manages courses, assignments and assessment for a school with four user roles - student, lecturer, Head of Department (HoD) and administrator - each restricted to the actions its role permits. Its defining control is moderation: a mark becomes visible to a student only after the HoD has approved and released it. The application runs as a single C++17 program over a local SQLite database on one school computer or local network, fully offline, for up to 800 students.

The purpose of this plan is to establish the project's feasibility, its work breakdown and schedule, and the risks to be managed at each phase gate. It covers Phase 1 only. The later work packages (the assignment's Phases 2–8, the professional/legal/ethical analysis, and the final report and viva) appear in the work breakdown and schedule as planned future work; no part of them is presented as complete.

Online payment, video conferencing, automated grading, plagiarism detection and web or mobile interfaces are out of scope and are excluded from every estimate below.

2. Feasibility Study

The project is feasible on all five dimensions assessed below. The binding constraint is schedule, not technology; it is managed through controlled scope and formal phase-gate reviews.

2.1 Technical Feasibility

The toolchain - C++17, SQLite, CMake and a unit-testing framework (GoogleTest/Catch2) - is mature, free and offline-capable, and is well suited to a single-machine application. It maps directly onto the case-study constraints: offline operation, old hardware, a single school computer or local network, up to 800 students, persistent storage, password security, role-based access, mark-change auditing and manageable deployment. The demanding parts - a data-access layer with parameterized queries, salted password hashing, an audit log and role-based access - are standard and well documented.

Rejected alternative - a client-server RDBMS (PostgreSQL or MySQL): this was rejected because the **case study requires offline operation on old hardware on a single machine**. A server database adds an always-on process plus configuration and administration, with no benefit at this scale, and weakens the offline/old-hardware fit. SQLite's embedded, file-based model is the correct choice.

The suitability of SQLite for up to 800 students is an engineering judgement based on the read-mostly, single-machine access pattern this system exhibits; it is presented as reasoning, not as an empirically measured performance guarantee.

My Verdict: FEASIBLE.

2.2 Operational Feasibility

Operational feasibility follows from the stakeholder statements. Teachers are not IT experts, so the interface must be simple - met by role-specific menus. Internet is unreliable, so the system operates offline. Routine operation - one machine, local backup of a single database file - is within a school technician's capacity, supported by the installation guide and per-role user manuals produced in Phase 7. No operational infrastructure beyond what the case study specifies is assumed.

My Verdict: FEASIBLE, but subject to the stated usability and operational constraints.

2.3 Economic Feasibility

Development uses free and open-source tooling on existing hardware, by one student developer, over approximately 180–210 developer-hours across the seven weeks (about 26–30 hours per week). Direct tooling and project cost is therefore minimal; the principal input is developer effort. This is a feasibility assessment for the assignment scenario, not a full commercial business case, so no rand figures are stated and a real school deployment is not claimed to carry literally zero total cost.

The benefit anchor is operational: a reduction in the administrative effort spent on manual mark recording and release, and a reduced risk of mark-recording errors and disputes through controlled, audited handling. Weighed against the modest effort input, the cost–benefit balance for the scenario is positive.

Verdict: FEASIBLE.

2.4 Legal Feasibility

The system processes learners' personal data and assessment records, so it is assessed against the applicable Lesotho legislation, the *Data Protection Act, 2012 (Act No. 5 of 2012)*. The Act establishes the Data Protection Commission as the statutory enforcement authority.

Institutional enforcement capacity remains a practical constraint: as reported by the Lesotho News Agency on 21 April 2026, the Data Protection Commission had not yet been established. That report quotes **Advocate 'Makamohelo Mofilikoane, Chief Legal Officer, Ministry of Information, Communication, Science, Technology and Innovation,** to that effect; this is the most recent available reporting. The planned system design will address the Act's data-protection principles regardless of the current state of institutional enforcement.

The LMS controls map onto the Act as follows:

- lawful processing and data minimization - only the learner and assessment data the system needs is collected and stored;
- security of the integrity of personal information (section 20) - salted password hashing and controlled access protect stored data;
- role-based access - restricting which role may see which learner data;
- auditability - every mark change is recorded (who, when, old value, new value, reason);
- sensitive categories (section 29) - section 29 regulates specified sensitive categories, including information concerning a child subject to parental control, and categories such as health, religion, race and political affiliation. It does not make every assessment record "sensitive"; the design nonetheless takes particular care where learner records fall within these categories, especially information concerning children subject to parental control;
- offline, local storage - keeping data on a single machine limits exposure.

Legal feasibility is conditional on implementing these controls; the plan makes no blanket claim that the system "is compliant." **Verdict: FEASIBLE WITH SPECIFIED CONTROLS.**

2.5 Schedule Feasibility

The seven-week Waterfall schedule is achievable but tight. Implementation cannot start before the design passes Gate 2; the critical path (Section 4.3) carries no usable slack, so any upstream delay moves the downstream milestones; and the 180–210-hour estimate (about 26–30 hours per week) is realistic only with disciplined scope control. The seven-week horizon is fixed by the assignment and is not altered.

Verdict: FEASIBLE BUT TIGHT.

2.6 Overall Recommendation

Recommendation: PROCEED. The principal constraint is schedule and time, not technology. Feasibility depends on controlled scope, adherence to the phase gates, disciplined scheduling, and active management of the high-exposure risks in Section 5. On those conditions, the project should proceed.

Summary of feasibility verdicts:

Dimension	Assessment	Verdict
Technical	Mature offline toolchain; SQLite fits single-machine scale	Feasible
Operational	Simple role-based menus; offline; technician-maintainable	Feasible (with constraints)
Economic	Free tooling; ~180–210 developer-hours; lower admin effort and mark-error risk	Feasible
Legal	Data Protection Act, 2012 controls specified (s20, s29, hashing, RBAC, audit)	Feasible with specified controls
Schedule	Bounded scope, but the critical path has no usable slack	Feasible but tight

3. Work Breakdown Structure

The project decomposes into ten work packages. Package 1 is the current Phase 1 work; packages 2–10 are planned future work. Each package is coded for traceability to the schedule (Section 4) and the risk register (Section 5). Week allocations match the Excel schedule master exactly.

WBS ID	Work Package	Key Output / Deliverable	Week(s)	Gate / Dependency
1	Project Planning & Management	This Project Management Plan (feasibility, WBS, schedule, risks)	Week 1	-
2	Requirements Engineering	SRS; use-case model; stakeholder/conflict analysis	Weeks 1–2	After 1 -> Gate 1
3	System Design	Architecture; class & behavioral models; DB design	Weeks 2–3	After 2 -> Gate 2
4	Implementation	Working C++17 / SQLite application	Weeks 4–5	After Gate 2
5	Verification & Validation	Test plan, test results, defect log	Weeks 5–6	After 4 -> Gate 3
6	Configuration Management & QA	Git workflow; CI pipeline; static analysis; QA records	Weeks 4–7 (parallel)	Parallel, from 4 (SS)
7	Deployment	v1.0 release; deployment diagram; install guide; manuals	Week 6	After Gate 3
8	Maintenance	CR1 impact analysis; v1.1 release; regression tests	Week 7	After 7
9	Professional, Legal & Ethical Analysis	Analysis section for the final report	Weeks 5–7 (parallel)	Parallel, from 1 (SS)
10	Final Report & Viva	Compiled report (traceability matrix); viva	Weeks 5–7 (parallel)	Parallel, from 1 (SS)

Packages 2–10 are shown as planned future work. No gate, test result or release for the later work packages has yet occurred.

4. Project Schedule

The schedule spans seven weeks (Week 1 to Week 7) and is expressed in week numbers, not calendar dates. It is phase-gated: each phase ends in a reviewed deliverable, and implementation cannot begin until the design passes Gate 2. Adjacent phases are fast-tracked where the seven-week envelope requires it, and configuration management/QA and the report work run in parallel from Weeks 4 and 5 respectively. **The Gantt below is exported from the Excel schedule master which I have appended/attached below.**

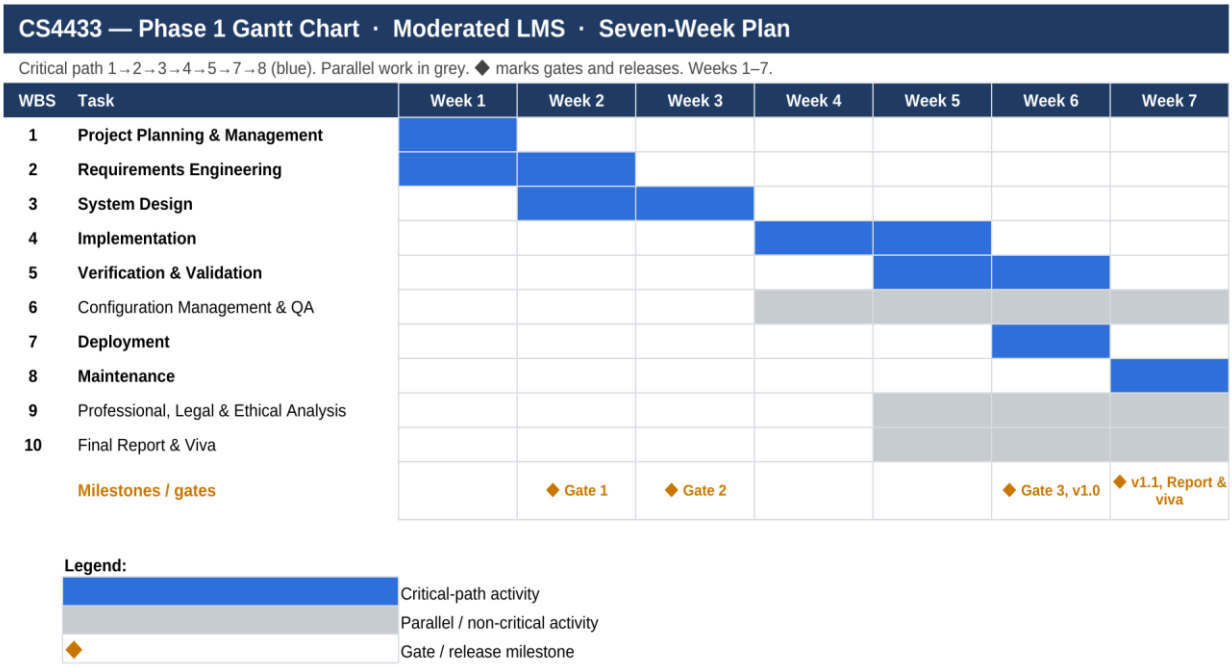


Figure 1. Phase 1 Gantt chart (exported from the Excel schedule master). Critical-path activities in blue; parallel work in grey; ◆ gates and releases.

4.1 Dependencies

Dependencies are finish-to-start (FS) along the main delivery chain and start-to-start (SS) for the parallel packages, enforced by the gates.

The week bars show each phase's allocated window, not the moment work hands over. The handoff is governed by the dependency and its phase gate rather than by the calendar week, so Requirements can occupy Weeks 1–2 while Design occupies Weeks 2–3 without Design beginning before the approved requirements baseline; likewise, Verification and Validation occupies Weeks 5–6 while Deployment is allocated to Week 6, because Gate 3 closes V&V within Week 6 before deployment and release work begins. The main chain 1 -> 2 -> 3 -> 4 -> 5 -> 7 - 8 and the hard Gate 2 dependency (design must pass Gate 2 before implementation) are unchanged.

Task	Predecessor	Dependency type	Reason
2 Requirements	1	FS	Planning fixes scope before requirements are elicited
3 System Design	2 (Gate 1)	FS	Design follows approved requirements
4 Implementation	3 (Gate 2)	FS	Build cannot begin until design passes Gate 2 (hard gate)
5 Verification & Validation	4	FS	Verification needs implemented code
7 Deployment	5 (Gate 3)	FS	Release only after V&V passes Gate 3
8 Maintenance	7	FS	CR1 / v1.1 follows the v1.0 release
6 Config. Mgmt. & QA	4	SS	Runs in parallel once implementation starts
9 Prof./Legal/Ethical	1	SS	Analyzed in parallel; folded into the report
10 Final Report & Viva	1	SS	Written throughout; viva at the close

4.2 Milestones

The milestones below use week numbers only; the acceptance column states the event that closes each one.

Milestone	Week	Acceptance / event
Gate 1 - Requirements approved	End of Week 2	Requirements baseline approved
Gate 2 - Design approved (build unlocked)	End of Week 3	Design approved; implementation authorised
Gate 3 - V&V complete	Week 6	Verification and validation complete
v1.0 release	End of Week 6	First release deployed
v1.1 release (CR1)	End of Week 7	Change request CR1 incorporated
Final report & viva	End of Week 7	Report submitted; viva

4.3 Critical Path

The critical path is 1 -> 2 -> 3 -> 4 -> 5 -> 7 -> 8 - planning, requirements, design, implementation, verification and validation, deployment and maintenance. These activities form the main sequential delivery chain; every gate lies on it; and it carries no usable slack. Configuration management/QA (6) and the report and analysis (9, 10) run in parallel and are not on the critical path. Because there is no slack, any delay to an upstream phase pushes every downstream milestone, including the v1.0 and v1.1 releases - which is why scope control and the gate reviews are the primary schedule defenses. To fit the fixed seven-week envelope, adjacent critical activities are fast-tracked: requirements and design overlap in Week 2, implementation and verification in Week 5, and verification, Gate 3 and deployment compress into Week 6. This overlap is what removes the slack and makes disciplined gate sign-off before each successor essential.

5. Risk Management

5.1 Risk Method

Each risk is scored on a 1–5 scale for likelihood and impact (1 = very low, 5 = very high). Exposure is their product (likelihood × impact, range 1–25). An exposure of 12 or more is treated as high and receives explicit attention at the next gate review. The register spans technical, project and people risks.

5.2 Phase 1 Baseline Risk Register

ID	Category	Risk	L	I	Exp	Mitigation	Owner
R1	Technical	SQLite data-access layer and parameterised queries take longer than estimated, slipping implementation	3	3	**9**	Build the data-access layer first in Week 4 as a thin spike; one shared prepared-statement wrapper reused everywhere	Dev
R2	Technical	Password-hashing library (e.g. Argon2/bcrypt) is hard to integrate with CMake	2	4	**8**	Choose a well-documented, header-light library early and prototype it in Week 4; keep a vetted fallback	Dev
R3	Technical	Artefacts drift - code diverges from the design, or the database no longer matches the class diagram - losing traceability marks	3	4	**12**	Maintain the traceability matrix continuously; update design docs on every change; consistency checked at each gate	Dev
R4	Project	Seven-week critical path overruns; Gate 2 blocks the start of implementation	4	4	**16**	Strict scope control (honor the out-of-scope list); timebox each phase; stand up CI early; write the report in parallel	Dev / Supervisor
R5	Project	Scope creep - adding out-of-scope features or gold-plating	3	3	**9**	Freeze the MoSCoW Must set at Gate 1; defer new ideas to a backlog; CR1 is the only sanctioned change	Dev
R6	People	Single developer: illness, competing coursework or exams reduce available hours	3	4	**12**	Front-load critical tasks; protect weekly hours; commit small and often so partial progress survives	Dev
R7	People	Viva under-preparation - marks withheld for components that cannot be explained	2	5	**10**	Write design rationale as work proceeds; keep a decisions log; rehearse the demo against the use cases before Week 7	Dev
R8	Technical	CI pipeline (building C++/CMake and running tests) is slow to set up	3	2	**6**	Use a standard CMake + test-framework workflow template; stand CI up in Week 4 on a minimal build	Dev

Scale: 1 = very low, 5 = very high. Exposure = likelihood × impact. Exposures of 12 or more (R3, R4, R6) are highlighted and tracked as high-priority at the next gate.

5.3 Risk Review and Gate Updates

This register is the Phase 1 baseline. It is reviewed and re-versioned at each phase gate - Gate 1 (requirements), Gate 2 (design) and Gate 3 (verification and validation). At each gate the register is checked for new risks, closed risks, and changed likelihood, impact, exposure, mitigation or ownership. This defines the review process; it records no future scores, because those gates have not yet occurred.

6. Assumptions and Constraints

The plan rests on the assumptions and constraints below. Assumptions are things taken to be true for planning; constraints are fixed limits imposed by the assignment or the environment.

Assumptions:

- One developer, myself (**Mweli Pama, 201404452**) for the full seven weeks.
- Existing hardware is used; no procurement.
- All tooling is free and open-source.
- Developer effort is approximately 180–210 hours (about 26–30 hours per week).
- Deployment target is one school computer or local network, offline, for up to 800 students.
- The out-of-scope exclusions (payments, video, auto-grading, plagiarism, web/mobile) remain in force.

Constraints:

- Seven-week schedule, fixed by the assignment.
- Implementation may not begin until the design passes Gate 2.
- The system must run offline on old hardware.
- Passwords are stored only as salted hashes.
- Every mark change is written to an audit log.

7. Gate-Review Governance

The project uses three formative gate reviews with the supervisor: Gate 1 (requirements), Gate 2 (design - which unlocks implementation) and Gate 3 (verification and validation). At each gate the plan calls for three actions: confirm the phase deliverable; re-score the risk register (Section 5.3); and re-check artefact consistency against the traceability matrix. The gates are the synchronization points at which the plan is formally updated; they are planned events, and none has yet occurred.

References

Data Protection Act, 2012 (Act No. 5 of 2012) (Lesotho). https://lesotholii.org/akn/ls/act/2012/5

Lesotho News Agency. (2026, April 21). *Urgent gaps in cyber security, data protection outlined.* https://www.lena.gov.ls/urgent-gaps-in-cyber-security-data-protection-outlined/